

DEPI R4 – Data Engineering Track

Project 5: Building a Retail Data Pipeline

Introduction

Requirement gathering is a critical phase in software development that focuses on identifying and documenting user needs and system expectations. Among the most important outputs of this phase are user stories and non-functional requirements, which together provide a complete understanding of system functionality and quality.

User stories represent system requirements from the user's perspective, describing what the user needs to achieve, and the value gained from each feature. On the other hand, non-functional requirements define how the system should perform, ensuring that it meets quality standards such as performance, security, and usability.

By integrating both user stories and non-functional requirements, developers can design systems that are not only functional but also efficient and reliable.

1. System Actors (Stakeholders)

Data Engineer (Primary Actor)

Role: Architect and builder of the pipeline.

Responsibility: Designs and implements data ingestion, writes cleaning scripts, manages transformations, and ensures Airflow DAGs run correctly.

Data Analyst

Role: Primary consumer of processed data.

Responsibility: Builds dashboards and performs SQL queries using the Data Warehouse to analyze sales trends.

Business Manager

Role: Decision-maker.

Responsibility: Reviews reports and KPIs to support strategic decisions such as inventory and marketing.

System Administrator

Role: Infrastructure support.

Responsibility: Manages system environment (Docker, databases, cloud), ensures security and uptime.

Requirement Gathering for User Stories

User stories are gathered by focusing on user interactions with the system and identifying the tasks users need to perform. The goal is to translate user needs into simple, clear, and actionable requirements.

Process of Gathering User Stories

The process involves:

Identifying stakeholders: Understanding who will use the system (data analysts, managers, and engineers)

Understanding user goals: what each user wants to achieve from the system.

Defining system interactions: how users interact with system features.

Writing user stories: Converting jobs into a simple sentence to describe what is required.

A graphic illustrating the User Story format. It features a large, light gray rectangular area with a blue square and a dotted pattern in the top-left corner and a blue square with diagonal lines in the bottom-right corner. In the center, the title "User Story" is written in a large, bold, black serif font. Below the title, on the left, is a yellow sticky note pinned with a blue pushpin. The sticky note contains the template: "As a <role> I want <goal> so that <benefit>" followed by "Acceptance criteria:" and "(Conditions of Satisfaction)" with two lines of ellipses. To the right of the sticky note, the same template is filled out with an example: "As an Account Manager I want a sales report of my account to be sent to my inbox daily So that I can monitor the sales progress of my customer portfolio". Below this example, the "Acceptance criteria:" section is followed by a numbered list of three items: "1. The report is sent daily to my inbox", "2. The report contains the following sales details: ...", and "3. The report is in csv format.".

User Story

As a <role>
I want <goal>
so that <benefit>

Acceptance criteria:
(Conditions of Satisfaction)
...
...

As an Account Manager
I want a sales report of my account to be sent to my inbox daily
So that I can monitor the sales progress of my customer portfolio

Acceptance criteria:

1. The report is sent daily to my inbox
2. The report contains the following sales details: ...
3. The report is in csv format.

2. User Stories

The following user stories are written from each stakeholder's perspective using the standard format: **As a <role>, I want <goal>, so that <benefit>**. Each story includes Acceptance Criteria that define the conditions of satisfaction.

Data Engineer

US-01: As a Data Engineer, I want to schedule automated daily ingestion from SQL databases and Parquet files, so that the pipeline runs without manual intervention.

Acceptance Criteria:

1. Ingestion runs daily at a configured time via Airflow DAG
2. System extracts data from both SQL and Parquet sources
3. Failed runs trigger alert notifications
4. Logs are stored for each run

US-02: As a Data Engineer, I want to apply data cleaning rules (remove duplicates, fix nulls, standardize formats), so that only high-quality data enters the warehouse.

Acceptance Criteria:

1. Duplicate records are removed before loading
2. Null values are handled according to defined rules
3. Data types are validated and corrected
4. Cleaning summary report is generated per run

US-03: As a Data Engineer, I want to transform raw data into a Star Schema (fact & dimension tables), so that analysts can run efficient queries.

Acceptance Criteria:

1. Fact table contains sales transactions with correct foreign keys
2. Dimension tables include Date, Product, Customer, and Supplier
3. Computed metrics (total_sales, quantity_sold) are stored correctly
4. Star Schema passes referential integrity checks

Data Analyst

US-04: As a Data Analyst, I want to query the Data Warehouse using SQL, so that I can analyze sales trends and generate business insights.

Acceptance Criteria:

1. Data Warehouse is accessible via standard SQL client
2. Query response time is under 10 seconds for standard reports
3. Processed data is refreshed daily by 6:00 AM

US-05: As a Data Analyst, I want to connect BI tools (e.g., Power BI) to the warehouse, so that I can build interactive dashboards.

Acceptance Criteria:

1. BI tool can connect via JDBC/ODBC
2. Dashboard reflects latest data after daily refresh
3. Dashboards support filtering by date, product, and region

Business Manager

US-06: As a Business Manager, I want to view daily KPI reports (revenue, top products, regional sales), so that I can make informed strategic decisions.

Acceptance Criteria:

1. KPI report is available every morning
2. Report includes total revenue, units sold, and top 10 products
3. Report is exportable as PDF or Excel

US-07: As a Business Manager, I want to monitor inventory levels through the dashboard, so that I can prevent stockouts and overstock situations.

Acceptance Criteria:

1. Inventory levels are visible per product and location
2. Low-stock alerts are triggered at defined thresholds
3. Historical inventory trends are available for the past 12 months

System Administrator

US-08: As a System Administrator, I want to monitor pipeline health and receive failure alerts, so that I can ensure system uptime.

Acceptance Criteria:

1. Airflow dashboard shows DAG status in real-time
2. Alerts are sent on DAG failure within 5 minutes
3. Logs are retained for at least 30 days

US-09: As a System Administrator, I want to manage access permissions for each user role, so that sensitive data is protected.

Acceptance Criteria:

1. RBAC is enforced across all components
2. Analysts can only read processed data, not raw staging tables
3. All access changes are logged with timestamps

3. Functional Requirements

Automated Data Ingestion

The system shall automatically extract data from SQL databases and Parquet files on a scheduled basis.

Data Staging & Storage

The system shall store raw data in a staging area for traceability and recovery before processing.

Data Cleansing

The system shall clean data by removing duplicates, handling missing values, correcting data types, and standardizing formats.

Data Transformation (ETL)

The system shall transform raw data into a Star Schema including fact and dimension tables and compute key business metrics such as total sales.

Workflow Orchestration

The system shall use Apache Airflow to schedule, monitor, and manage pipeline tasks with logging and alerting.

Data Loading (Warehouse)

The system shall load processed data into a centralized Data Warehouse optimized for querying.

Data Quality Validation

The system shall validate data using rules such as uniqueness, completeness, and value ranges.

Analytics & Reporting Support

The system shall provide data access to BI tools with scheduled or near real-time refresh for reporting.

Requirement Gathering for Non-Functional Requirements

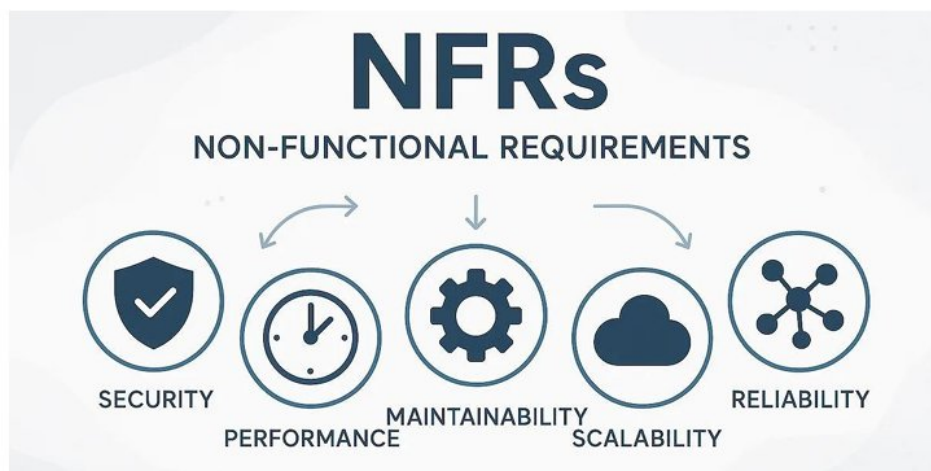
What Is a Non-Functional Requirement?

A non-functional requirement defines the specifications that describe a system's constraints or operational abilities.

It also describes criteria used to judge a system's operations and doesn't refer to the behaviors or functions within that system.

Software engineers and requirements analysts consider nonfunctional requirements (NFRs) as the limits and capabilities of system behavior.

A non-functional requirement also acts as a system quality that designs solutions based on constraints and abilities.



Process of Gathering Non-Functional Requirement

The process includes:

Analyzing system constraints:

Identifying system limitations such as capacity, resources, and technical restrictions.

Identifying quality attributes:

Determining key attributes such as performance, security, usability, and reliability.

Understanding user expectations:

Defining how fast, secure, and easy the system should be from the user's perspective.

Documenting standards:

Establishing clear and measurable criteria for system performance and behavior.

Non-Functional Requirement Types

Non-functional requirement types are also called the "ity" requirements.

Ultimately, non-functional requirements should include a detailed description of the following features:

- **Availability/Reliability** – What is the normal usage critical failure time, and do users need 24/7 access?
- **Capacity** – The system storage requirements for today and the future, and how will it scale?
- **Compatibility** – The minimum hardware requirements and which operating systems/versions on which devices must it support?
- **Maintainability/Manageability** – The time it takes to repair components and the ease of administrator system management.
- **Performance** – A quality attribute that oversees the system's responsiveness to multiple user interactions to avoid a negative user experience.
- **Recoverability/Serviceability** – The time it takes the system to recover from critical failures and the time between critical system events.
- **Scalability** – Another quality attribute that defines high system loads where the system still performs.
- **Security** – How the system stores and shares sensitive information, an IT department's adherence to specific standards, and the security best practices to protect sensitive data.
- **Usability** – The product's ease of use and a definition of the intended user experience (UX).

Examples of Non-Functional Requirements

The system should respond within a few seconds.

The system must secure user data such as passwords and IDs.

The system should support multiple devices and platforms.

The system should operate without frequent failures.

These challenges highlight the importance of continuous communication and validation

Importance of Requirement Gathering

Requirement gathering for both user stories and non-functional requirements is essential because it:

- Ensures the system meets user expectations
- Defines both functionality and system quality
- Reduces development errors and rework
- Improves system performance and usability
- Provides a clear structure for development.